Group #7

Members:

Kristine Nguyen

Fernanda Lugo

Xuan Phat Tran

- ask the users to enter a number in binary or decimal. The number is unsigned.
- ask the users what type of number it is (binary or decimal)

```python
#Kristine Nguyen: Input Collection and Output Display

# Ask the user to enter the number
number = input("Enter the number (only unsigned numbers): ")
# Ask the user what type of number they are entering
number_type = input("Enter number type: ").lower()

# Check if the number is legal and convert it to the other 3 types of numbers.
if is_legal_number(number, number_type):
    result = convert_number(number, number_type)
    print(f"Decimal: {result['decimal']}")
    print(f"Binary: {result['binary']}")
    print(f"Octal: {result['octal']}")
    print(f"Hexadecimal: {result['hexadecimal']}")
else:
    print(f"Error: '{number}' is not a legal {number_type} number.")
```

- check to make sure the number is legal (a decimal number contains only 0-9). If the number is not legal, it should error out!

```python
#Fernanda Lugo: Validation Function - Legal Number Check
# Checks whether the number entered is a legal decimal or binary number
def is_legal_number(number, number_type):
    decimal_points = 0
    digit_count = 0

    # The input cannot be empty
    if number == "":
        return False

    for character in number:

        # Allow only one decimal point for floating-point numbers
        if character == ".":
            decimal_points += 1

            if decimal_points > 1:
                return False

        # Decimal numbers can only contain digits 0 through 9
        elif number_type == "decimal":
            if character < "0" or character > "9":
                return False

            digit_count += 1

        # Binary numbers can only contain 0 and 1
        elif number_type == "binary":
            if character != "0" and character != "1":
                return False

            digit_count += 1

        # Reject an invalid number type
        else:
            return False

    # Reject an entry containing only a decimal point
    if digit_count == 0:
        return False

    return True
```

- If the number is legal, convert the number to the other 3 types (Hint: There are a total of 4 types: binary, octal, hexadecimal, decimal)

```python
#Xuan Phat Tran: Base Conversion Functions
HEX_DIGITS = "0123456789ABCDEF"

# Function to convert a binary string to decimal
# This function takes a binary string (which may include a fractional part) and converts it to its decimal equivalent.
# Decimal type would be used as a hub for all conversions, as it can represent both integer and fractional values.

def binary_str_to_decimal(binary_str):
    """Convert a binary string (optionally with a '.' fraction) to decimal."""
    if '.' in binary_str:
        int_part, frac_part = binary_str.split('.')
    else:
        int_part, frac_part = binary_str, ''

    # integer part: sum of bit * 2^position
    decimal_value = 0
    power = len(int_part) - 1
    for bit in int_part:
        decimal_value += int(bit) * (2 ** power)
        power -= 1

    if not frac_part:
        return decimal_value

    # fractional part: sum of bit * 2^-position
    frac_value = 0.0
    power = 1
    for bit in frac_part:
        frac_value += int(bit) * (2 ** -power)
        power += 1

    return decimal_value + frac_value
```

```python
# The following functions convert decimal numbers to binary, octal, and hexadecimal using the division/multiplication method.
# The number of digits after the decimal point can be specified for each base.
# In this implementation, the decimal_to_base function handles the conversion for any base (2, 8, or 16) and is called by the specific conversion functions for binary, octal, and hexadecimal.
# So that we don't have to repeat the same logic for each base conversion, we can use a single function that takes the base as a parameter.


def decimal_to_base(decimal_value, base, digits_after_point):
    """Division/multiplication method, works for base 2, 8, or 16."""
    int_part = int(decimal_value)
    frac_part = decimal_value - int_part

    # integer part: repeated division, collect remainders
    if int_part == 0:
        int_digits = "0"
    else:
        digits = []
        n = int_part
        while n > 0:
            digits.append(HEX_DIGITS[n % base])
            n //= base
        int_digits = "".join(reversed(digits))

    if frac_part == 0:
        return int_digits

    # fractional part: repeated multiplication, collect integer bits
    frac_digits = []
    f = frac_part
    for _ in range(digits_after_point):
        f *= base
        digit = int(f)
        frac_digits.append(HEX_DIGITS[digit])
        f -= digit

    return int_digits + "." + "".join(frac_digits)
```

```python
def decimal_to_binary(decimal_value):
    return decimal_to_base(decimal_value, 2, digits_after_point=16)


def decimal_to_octal(decimal_value):
    return decimal_to_base(decimal_value, 8, digits_after_point=6)


def decimal_to_hexadecimal(decimal_value):
    return decimal_to_base(decimal_value, 16, digits_after_point=4)

# This function would be called after the legality check of the input number. It takes a validated number string and its type (binary or decimal) and converts it to all four representations: decimal, binary, octal, and hexadecimal.
def convert_number(num_str, num_type):
    """
    num_str: validated number as a string (legality check happens elsewhere)
    num_type: 'binary' or 'decimal'
    """
    if not is_legal_number(num_str, num_type):
        raise ValueError(f"Illegal {num_type} number: {num_str!r}")

    if num_type == 'binary':
        decimal_value = binary_str_to_decimal(num_str)
    elif num_type == 'decimal':
        decimal_value = float(num_str) if '.' in num_str else int(num_str)
    else:
        raise ValueError("num_type must be 'binary' or 'decimal'")

    return {
        'decimal': decimal_value,
        'binary': decimal_to_binary(decimal_value),
        'octal': decimal_to_octal(decimal_value),
        'hexadecimal': decimal_to_hexadecimal(decimal_value)
    }
```

Decimal Cases

Legal

```
Enter the number (only unsigned numbers): 12345.6789
Enter number type: decimal
Decimal: 12345.6789
Binary: 11000000111001.1010110111001100
Octal: 30071.533461
Hexadecimal: 3039.ADCC
```

```
Enter the number (only unsigned numbers): 12345
Enter number type: decimal
Decimal: 12345
Binary: 11000000111001
Octal: 30071
Hexadecimal: 3039
```

```
Enter the number (only unsigned numbers): 0.1111
Enter number type: decimal
Decimal: 0.1111
Binary: 0.0001110001110001
Octal: 0.070704
Hexadecimal: 0.1C71
```

Illegal

```
Enter the number (only unsigned numbers): 1.2.3.4
Enter number type: decimal
Error: '1.2.3.4' is not a legal decimal number.
```

Only 1 dot in a decimal number

```
Enter the number (only unsigned numbers): ABCD
Enter number type: decimal
Error: 'ABCD' is not a legal decimal number.
```

ABCD is not a decimal number

Binary Cases

Legal

```
Enter the number (only unsigned numbers): 10101010
Enter number type: binary
Decimal: 170
Binary: 10101010
Octal: 252
Hexadecimal: AA
```

```
Enter the number (only unsigned numbers): 100101.110010101011
Enter number type: binary
Decimal: 37.791748046875
Binary: 100101.1100101010110000
Octal: 45.625300
Hexadecimal: 25.CAB0
```

```
Enter the number (only unsigned numbers): 101010100000.111101010101
Enter number type: binary
Decimal: 2720.958251953125
Binary: 101010100000.1111010101010000
Octal: 5240.752500
Hexadecimal: AA0.F550
```

Illegal

```
Enter the number (only unsigned numbers): 123
Enter number type: binary
Error: '123' is not a legal binary number.
```

Wrong type using

```
Enter the number (only unsigned numbers): $
Enter number type: binary
Error: '$' is not a legal binary number.
```

This is not a number

Additional Error Cases

A is both not a decimal or binary number, so it is illegal to input in this program, leading to an error.

```
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System> & C:\Users\lenovo\AppData\Local\Programs\Python\Python313\pytho
n.exe c:/Users/lenovo/Desktop/nextgen/Lab-1-Computer-System/Lab_1_Assignment.py
Enter the number (only unsigned numbers): A
Enter number type: decimal
Error: 'A' is not a legal decimal number.
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System>
```

```
PS C:\Users\lenovo\Desktop\nextgen\Lab-1-Computer-System> & C:\Users\lenovo\AppData\L
n.exe c:/Users/lenovo/Desktop/nextgen/Lab-1-Computer-System/Lab_1_Assignment.py
Enter the number (only unsigned numbers): 5
Enter number type: binary
Error: '5' is not a legal binary number.
```

5 is legal if users input type as "decimal", but it could not be defined as binary because binary only allow 0 and 1.

Extra Running Example

```
Enter the number (only unsigned numbers): 125.95
Enter number type: decimal
Decimal: 125.95
Binary: 1111101.1111001100110011
Octal: 175.746314
Hexadecimal: 7D.F333
```

```
Enter the number (only unsigned numbers): 10110111000
Enter number type: binary
Decimal: 1464
Binary: 10110111000
Octal: 2670
Hexadecimal: 5B8
```